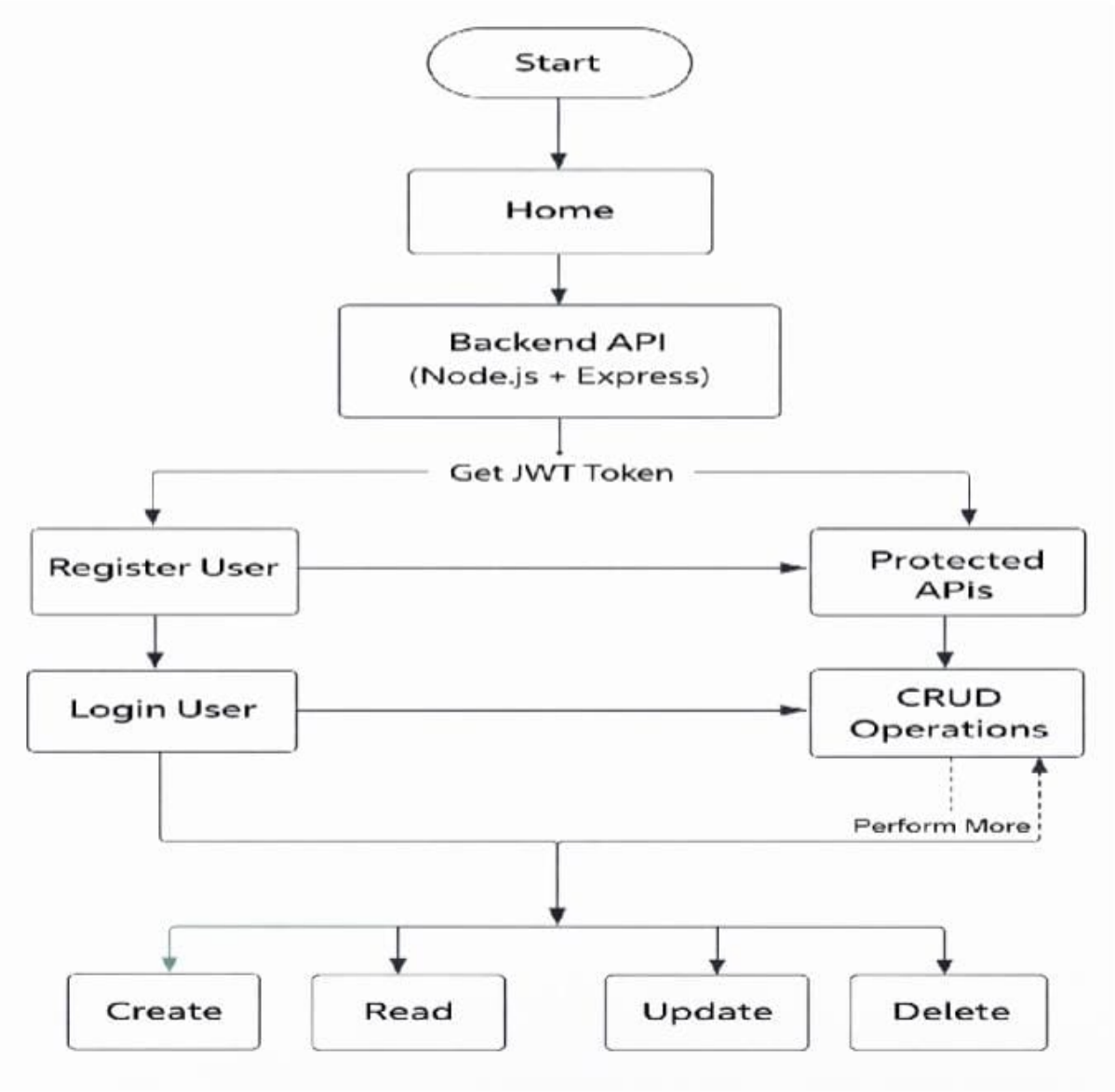
# Project Design Phase-II

## Data Flow Diagram & User Stories

| | |
|---|---|
| Date | 04 March 2026 |
| Team ID | SWTID-2026-3045 |
| Project Name | SpendWise |
| Maximum Marks | 4 Marks |

**Data Flow Diagrams:**



## User Stories:

| User Type | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|
| User (API User) | USN-1 | As a user, I can register using name, email, and password | User is able to create an account and receives a success response | High | Sprint-1 |
| User (API User) | USN-2 | As a user, I can login using email and password | User receives a valid JWT token after successful login | High | Sprint-1 |
| User (API User) | USN-3 | As a user, I can access protected APIs using JWT token | Requests without token are rejected; valid token allows access | High | Sprint-2 |
| User (API User) | USN-4 | As a user, I can create a new record/transaction | Record is saved in database and returned in response | High | Sprint-2 |
| User (API User) | USN-5 | As a user, I can view all my records/transactions | User receives only their own records from the system | High | Sprint-2 |
| User (API User) | USN-6 | As a user, I can update my record/transaction | Selected record is updated and new data is returned | Medium | Sprint-3 |
| User (API User) | USN-7 | As a user, I can delete my record/transaction | Selected record is removed and success message is returned | Medium | Sprint-3 |
| User (API User) | USN-8 | As a user, I cannot access other users' data | System blocks access to data that does not belong to the user | High | Sprint-3 |

**Functional Requirements:**

Following are the functional requirements of the proposed solution.

| FR No. | Functional Requirement (Epic) | Sub Requirement (Story / Sub-Task) |
|---|---|---|
| **FR-1** | **User Authentication** | User can register using name, email, and password |
| | | User can login using email and password |
| | | System generates JWT token after successful login |
| **FR-2** | **Secure Access** | User can access protected APIs only with valid JWT token |
| | | Middleware verifies JWT token before allowing access |
| **FR-3** | **Data Management (CRUD)** | User can create a new record/transaction |

| | | |
|---|---|---|
| | | User can view all their records/transactions |
| | | User can update their own record/transaction |
| | | User can delete their own record/transaction |
| **FR-4** | **User Data Isolation** | Each user can access only their own data |
| | | One user cannot see or modify another user's data |
| **FR-5** | **Database Integration** | System stores user and record data in MongoDB |
| | | System retrieves and updates data using Mongoose |

**Non-functional Requirements:**
Following are the non-functional requirements of the proposed solution.

| **FR No.** | **Non-Functional Requirement** | **Description** |
|---|---|---|
| NFR-1 | **Usability** | The APIs should be simple and easy to understand for beginners. The project structure should be clear and well-organized. |
| NFR-2 | **Security** | All protected APIs must require JWT authentication. User passwords must be stored in encrypted (hashed) form. |
| NFR-3 | **Reliability** | The system should work consistently without crashes and handle valid and invalid requests properly. |
| NFR-4 | **Performance** | API responses should be returned within a short time for normal requests under student-level usage. |
| NFR-5 | **Availability** | The backend server should be available whenever it is running locally or deployed, allowing users to access APIs. |
| NFR-6 | **Scalability** | The project structure should allow adding more features (like filters, summary, frontend) in the future without major changes. |
| NFR-7 | **Maintainability** | Code should be modular (controllers, routes, models, middleware) so it is easy for beginners to understand and modify. |

## Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

**Table-1 : Components & Technologies:**

| S.No | Component | Description | Technology |
|---|---|---|---|
| 1. | User Interface | API testing interface or optional web UI | Postman / Thunder Client / (Optional) React |
| 2. | Application Logic | Handles authentication, CRUD operations, and business logic | Node.js, Express.js |
| | Database Layer | Stores users and records/transactions | MongoDB, Mongoose |
| 4. | Authentication | Secures APIs using token-based authentication | JWT (JSON Web Token), bcrypt |
| 5. | Configuration | Environment variables and DB connection setup | dotenv, Node.js |
| 6. | API Layer | Exposes REST APIs for register, login, and CRUD operations | Express.js, REST APIs |

**Table-2: Application Characteristics:**

| S.No | Characteristics | Description | Technology |
|---|---|---|---|
| 1. | Open-Source Frameworks | Uses open-source backend frameworks and libraries | Node.js, Express.js, Mongoose |
| 2. | Secure Authentication | Protects routes using token-based authentication | JWT, bcrypt |
| 3. | Database Integration | Connects and performs operations on NoSQL database | MongoDB |
| 4. | RESTful Architecture | Follows REST principles for API design | HTTP, Express.js |
| 5. | Scalable Structure | Organized folder structure for controllers, models, routes, middleware | Node.js Project Structure |
| 6. | Environment Configuration | Keeps secrets and config outside code | dotenv (.env file) |